

Modern C++

Trainer: Nilesh Ghule

Modern C++ in the Real World

- Powering the software where performance, reliability, and control matter.
  - 🎮 Gaming & Graphics — Unreal Engine, real-time rendering, physics simulations
  - 🌐 Browsers & Systems Software — Chromium, JS engines, high-performance infra
  - 🚗 Automotive & Robotics — Autonomous driving, sensor processing, real-time control
  - 📈 Finance & Trading — Low-latency trading, risk analysis, financial simulations
  - ✨ AI & Scientific Computing — High-performance computing, CV, numerical simulation
  - 🏭 Embedded & IoT — Industrial controllers, IoT devices, resource-constrained systems

- Why Modern C++?
  - High performance • Efficient resource management • Hardware control
  - Abstractions without unnecessary runtime overhead
- Modern C++ (C++11/14/17/20/23 and evolving standards) combines low-level control with expressive, maintainable code.

Getting Started with Modern C++

- Modern C++ is the continued evolution of C++—not a separate language
- Goal: write programs that are safer, clearer, and easier to maintain
- Choose constructs that communicate intent and help the compiler detect mistakes

What Is Modern C++?

- C++ standards are released periodically, each adding language and library features
- **C++11**: Major modernization—auto, range-based for loops, move semantics, rvalue references, lambda expressions, variadic templates, initializer lists, nullptr, constexpr, type aliases, enum class, delegating and defaulted/deleted functions, override and final, uniform initialization, smart pointers, Enhanced STL (array, tuple, function), Multi-threading and concurrency support.
- **C++14**: Refinements and usability improvements— improved lambdas, return type deduction for functions, variable templates, binary literals, smart pointer support, and improved standard library utilities.
- **C++17**: Major language and library enhancements—structured bindings, if constexpr, fold expressions, inline variables, Standard library additions (optional, variant, any, string_view, filesystem, byte), CTAD, guaranteed copy elision, Standard library additions (apply, invoke, clamp), and parallel algorithms.
- **C++20**: Major language evolution—concepts and constraints, ranges and views, coroutines, modules, three-way comparison, designated initializers, consteval, constinit, expanded constexpr capabilities, abbreviated function templates, template lambdas, jthread, format(), calendar and time-zone facilities, STL additions (erase, erase_if, atomic_ref), and synchronization utilities.
- **C++23**: Further language and library refinements—print, generator, std::ranges::to, std::views, explicit object parameters, multidimensional subscripting, deducing this, if consteval, expanded library support for ranges, formatting, containers, and compile-time programming.

The Guiding Principle

- Prefer code that makes important decisions visible
  - nullptr clearly communicates 'this is a null pointer'; 0 is primarily an integer
  - const auto& communicates 'read-only, no copy' in a loop
- Not a rule that 'new syntax is always better'
  - If the type is important to understanding, spell it out
  - If braces make a declaration confusing, use parentheses or an explicit type
- The right choice is the one that is safe and understandable in context

Learning Objectives

- Create and build a C++ Console App in Visual Studio 2026 using C++20
- Use brace initialization to prevent common narrowing conversions
- Explain when auto improves readability and when an explicit type is clearer
- Distinguish nullptr from older null-pointer conventions NULL and 0
- Choose between auto, const auto, auto&, and const auto& in a range-based loop

Prepare Visual Studio 2026

- Install the 'Desktop development with C++' workload in Visual Studio Installer
- Create a new Empty Project or Console App project for C++
  - May choose from filters
  - Language: C++
  - Platform: Windows
  - Type: Console
- You may choose to keep solution and project in same directory.
- Use the default x64 target

Set the Language Standard

- Open Project → Properties
- Set Configuration to All Configurations and Platform to x64
- Navigate to Configuration Properties → C/C++ → Language → C++ Language Standard
- Select ISO C++ Standard
  - Default: /std:c++20
  - For newer features: /std:c++23preview
- Build with Ctrl+Shift+B; run with Ctrl+F5.

Compiler vs. IDE

- Visual Studio is the IDE; MSVC is the compiler toolset
- Participants may have different toolset versions or update levels
- When a sample fails, first verify the language-standard setting and installed toolset
- Course uses C++20 as baseline
- C++23 features like std::print require appropriate mode (c++23preview) and toolset support — Need not to use them for every example

Uniform Initialization

- C++ supports copy initialization, direct initialization, and direct-list initialization
- Uniform initialization uses braces {} for a consistent syntax across types
- Key advantage: list initialization rejects common narrowing conversions
  - Narrowing occurs when a conversion can lose information, e.g., double to int

```
int attempts = 3;   // Copy initialization
int timeout(10);    // Direct initialization
int retryCount{3}; // Direct-list initialization

int itemCount{3.14}; // Ill-formed: narrowing conversion
```

traditional C++ init syntax

Nilesh Ghule

Braces vs. Parentheses with Containers

- Braces are not interchangeable with every other initialization form
- Class types with std::initializer_list constructors can receive preference during list initialization
- Choose the form that expresses intended behavior clearly

```
#include <vector>

std::vector<int> first{3};  // One element with value 3
std::vector<int> second(3); // Three value-initialized ints (0, 0, 0)
```

Initializing Arrays and Strings

- Arrays can be initialized with braces; compiler deduces size from the number of elements
- If fewer values than size are provided, remaining elements are zero-initialized
- Empty braces value-initialize all elements to zero
- For char arrays, a string literal includes the terminating null character
- std::string can be initialized with braces from a string literal

```
int scores[]{75, 82, 90, 68}; // Four elements
int scores[4]{75, 82};        // Remaining two are 0
int scores[4]{};              // All four are 0
int *arr = new int[]{1, 2, 3}; // Dynamically allocated array with array initializer.

char greeting[]{"Hello"};    // Six elements (includes '\0')

#include <string>
std::string greeting{"Hello"}; // std::string containing "Hello"
```

Braced-Init-List and std::initializer_list

- A braced-init-list is a comma-separated list of initializers in braces
- Can provide values for elements or be an argument to a constructor accepting std::initializer_list
- Elements of std::initializer_list are immutable through that list
- A braced-init-list is not itself an ordinary expression with a standalone type

```
#include <initializer_list>
#include <iostream>

void printValues(std::initializer_list<int> values) {
    for (int value : values) {
        std::cout << value << ' ';
    }
}

int main() {
    printValues({10, 20, 30}); // Prints: 10 20 30
}
```

Prefer Clear Initialization

- Initialize at the point of declaration when there is no reason to delay
- Declaring first and assigning later can leave the variable uninitialized
- For constants, use const—it communicates intent and lets the compiler reject modification
- const applies to the object or access path declared with it, not automatically to other objects

```
int retryCount{3}; // Clear and initialized

int retryCount;    // Uninitialized
retryCount = 3;

const int passingScore{60}; // Cannot be modified
```

Use auto When the Type Is Clear

- auto asks the compiler to deduce a variable's type from its initializer
- The type is determined at compile time—auto does not make C++ dynamically typed
- Useful when type names are long: iterators, library return values, templates
- Avoid auto when the initializer hides a conversion or the intended type is not obvious

```
auto count = 42;       // int
auto average = 86.5;   // double

auto ratio = 1;        // int, not double
// Better if floating-point is intended:
double ratio = 1.0;
// or
auto ratio = 1.0;      // double
```

auto, const, and References

- When auto deduces a value type, top-level const and reference qualifiers are not preserved
- auto value = total; creates a copy
- auto& value = total; declares a reference to the existing object
- const auto& provides read-only access without copying

```
int total = 10;
const int& totalAlias = total;

auto copy = totalAlias;        // int: separate copy
const auto& readOnly = total; // const int&: read-only reference

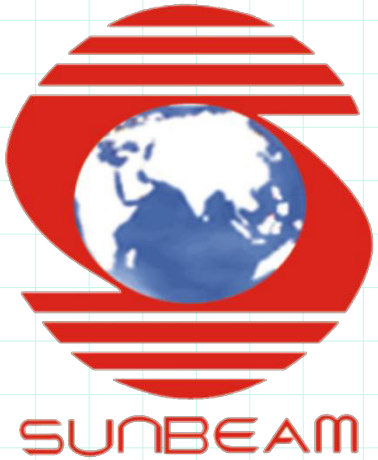
// copy can change without affecting total
// readOnly refers to total; cannot modify total through it
```

auto and Braces

- In C++17 and later, these deduce different types:
  - auto one = {1}; deduces std::initializer_list<int>
  - auto two{1}; deduces int
- Useful when reading existing code,
  but prefer simple initializers like auto two = 1;

```
auto one = {1}; // std::initializer_list<int>
auto two{1};    // int

auto three = 1; // int (clearer)
```



Thank You!

Nilesh Ghule

<nilesh@sunbeaminfo.com>